



Prerequisites for Monitoring

Ensure Coolify running with domain/subdomains (e.g., monitor.yourdomain.com).^[1]

Telegram alerts (recommended, free): @BotFather > New Bot > Copy token; chat with bot, get chat_id via /my_id or API.

Hetzner console: Cloud > Servers > Your VPS > "Enable Monitoring" (free pings).^[2]

VPS access for Netdata (SSH).

Step 1: Deploy Uptime Kuma (Endpoint Uptime)

Coolify UI > Applications > New > Marketplace > Search "Uptime Kuma" (louislam/uptime-kuma).

- Server: Hetzner-Primary.
- Domain: monitor.yourdomain.com (auto-SSL).
- Ports: 3001:3001.
- Env: PORT=3001.
Deploy (~1min).
Access <https://monitor.yourdomain.com> > Add Monitor:
 - HTTP: <https://api.yourdomain.com/health> (every 60s, alert if >5s).
 - TCP: Supabase `supabase-db.internal:5432`.
 - Keyword: FastAPI logs page.
Settings > Notifications > Telegram: Token/chat_id > Test.^[3] ^[1]

Step 2: Install Netdata (Real-Time Metrics + AI Anomalies)

SSH VPS:

```
bash <(curl -Ss https://get.netdata.cloud/kickstart.sh) --stable --disable-telemetry  
systemctl enable --now netdata-agent
```

Coolify > New > Docker Image > louislam/uptime-kuma wait no—Netdata dashboard:

- New > Docker Compose:

```
version: '3'  
services:  
  netdata:  
    image: netdata/netdata:latest  
    hostname: yourdomain.com # VPS hostname  
    ports: ['19999:19999']
```

```
volumes: ['/proc:/host/proc:ro', '/sys:/host/sys:ro', '/var/run/docker.sock:/var/run
environment:
  - NETDATA_CLAIM_TOKEN=optional
```

Deploy <https://netdata.yourdomain.com:19999>.

AI: Auto-baselines after 1h; alerts CPU>80%, OOM, disk>90% via Telegram webhook.^[4]

Step 3: Deploy Self-Hosted Sentry (Error Tracking + AI Grouping)

Coolify > New > Marketplace > "Sentry" (getsentry/self-hosted, 2GB RAM recommended).

- Server: Primary (or new CX22 if scaling).
 - Domain: sentry.yourdomain.com.
 - Env vars (post-deploy):
 - SENTRY_SECRET_KEY=gen_random_50chars
 - POSTGRES_DB=sentry, link to Supabase or new PG.
- Deploy (~10min init).
- Setup: <https://sentry.yourdomain.com> > Create org/project.
- Integrate code:
- FastAPI: `pip install sentry-sdk`, add:

```
import sentry_sdk
sentry_sdk.init(dsn="https://key@sentry.yourdomain.com/project", traces_sample_rate=1.0
```

Next.js: `npm i @sentry/nextjs, sentry.server.config.js`.

AI: Auto-groups similar errors, suggests fixes (e.g., "Redis timeout cluster").^[5]

Step 4: AI Log Anomaly Detection (Cordon)

Coolify > New > Docker Compose > "cordon" (open-source LLM logs).

`docker-compose.yml`:

```
services:
  cordon:
    image: ghcr.io/panaversity/cordon:latest
    ports: ['8080:8080']
    volumes: ['./logs:/logs'] # Mount Docker logs: docker logs -f app > /logs/app.log c
    environment:
      - MODEL=ollama/llama3.2 # Run Ollama separate for local LLM
```

First, deploy Ollama (marketplace): ollama.yourdomain.com, pull `llama3.2:1b`.

Pipe logs: `VPS cron */5 * * * * docker logs fastapi 2>&1 >> /path/to/logs/fastapi.log`.

Cordon dashboard detects anomalies (e.g., "sudden 5xx from BGE-M3"). Alerts via webhook to Telegram.^[6]

Step 5: Hetzner Native + Coolify Alerts

Hetzner Console > Monitoring: Add checks (HTTP api.yourdomain.com, ping IP); email/URL alerts.

Coolify > Settings > Notifications: Telegram/Slack for deploys/failures/disk full.

Dashboard: Coolify overview shows all services CPU/RAM/logs realtime.^{[1] [2]}

Verification & Optimization

- Simulate failure: `docker stop fastapi` > Check alerts in 1min.
 - Dashboards: Netdata family view per container; Uptime status page embed.
 - Scale: Multi-server? Netdata agents stream to central.
- Total overhead: 1GB RAM; €0 cost. Mobile: Telegram + Netdata app.^[4]



1. <https://coolify.io>
2. <https://docs.hetzner.com/robot/dedicated-server/security/system-monitor/>
3. <https://dev.to/disane/uptime-kuma-monitor-and-track-the-uptime-of-services-3a3o>
4. <https://www.netdata.cloud/solutions/technologies/hetzner-monitoring/>
5. <https://stackshare.io/stackups/sentry-vs-uptime-kuma>
6. <https://developers.redhat.com/articles/2025/12/09/semantic-anomaly-detection-log-files-cordon>